ChatGPT

แผนการอบรมเชิงปฏิบัติการ 2 วัน: Gemini RAG + SQL RAG + Agentic RAG สำหรับ Web App Chatbot

สรุปผู้บริหาร

รายงานนี้ออกแบบ “หลักสูตรอบรมเชิงปฏิบัติการ 2 วัน แบ่ง 5 เวิร์กช็อป” เพื่อให้ผู้เรียนสร้าง **Web App Chatbot** ที่ตอบคำถามจาก “ความรู้แบบเอกสาร (RAG)” และ “ข้อมูลเชิงโครงสร้างในฐานข้อมูล (SQL RAG)” โดยเน้นการใช้งาน **Gemini ผ่าน Google GenAI SDK** ซึ่งเป็นไลบรารีทางการที่เอกสารแนะนำและอยู่ในสถานะ GA พร้อมแนวทางย้ายจาก Gemini Developer API ไปใช้งานบน Vertex AI ได้โดยแนวคิด/โค้ดใกล้เคียงกันมาก (เหมาะกับเส้นทาง Prototype → Production) [1]

นอกจากนี้ หลักสูตรรวม “Agentic RAG” เพื่อเพิ่มความเชื่อถือได้ (reliability) ผ่านกลไก **routing, critique/evaluate, query rewrite และ tool-use** ด้วยแนวคิดเวิร์กโฟลว์ตัวแทน (agentic workflows) และการ orchestration แบบกราฟ (เช่น LangGraph ที่นิยาม state/nodes/edges) [2] พร้อมทั้งแนบตัวอย่างสถาปัตยกรรมด้วย Mermaid, โค้ดย่อ (backend, frontend, RAG pipeline, SQL retrieval, vector DB, prompt templates), แบบฝึกหัด–เฉลย, แผ่นงานผู้เรียน, เกณฑ์ประเมินผล, และแนวทาง deployment (hosting/security/scaling) บน Cloud Run + Cloud SQL + Secret Manager + Cloud Armor โดยอ้างอิงเอกสารทางการเป็นหลัก [3]

ข้อกำกับความสดของเอกสาร (doc freshness): เอกสาร Gemini API ที่ใช้อ้างอิงในรายงานนี้มี “Last updated” ชัดเจน เช่น Quickstart/Library/API Reference/Embeddings = 2026-01-19 UTC และ Function Calling = 2026-02-20 UTC (ตรวจสอบได้จากบรรทัดท้ายหน้าเอกสาร) [4]

สมมติฐาน ขอบเขต และผลลัพธ์ที่คาดหวัง

สมมติฐานที่ผู้ใช้ไม่ได้ระบุ (ระบุให้ชัดตามข้อกำหนด): - ระดับผู้เข้าอบรม: *unspecified* → ในแผนนี้สมมติ “ระดับกลาง” (เขียน Python/JS ได้, เข้าใจ HTTP/JSON, ใช้ Git/CLI ได้)
- จำนวนผู้เข้าอบรม: *unspecified* → สมมติ 20 คน (เหมาะกับการทำ lab แบบคู่/กลุ่มย่อย)
- ภาษาอบรม: ไทย (ตามคำขอ)
- เครื่องมือหลักที่ใช้สาธิต: Python + FastAPI (backend), React (frontend), PostgreSQL + pgvector (vector DB + SQL DB), Docker Compose (local infra)
- การเข้าถึงโมเดล: ใช้ Gemini ผ่าน Google GenAI SDK (API key) เป็นค่าเริ่มต้น; ส่วนโหมด Vertex AI (ADC/Project/Location) เป็น “เส้นทาง production” ให้ทำช่วงท้าย [5]

ขอบเขตความสามารถของระบบที่ผู้เรียนจะทำได้ภายใน 2 วัน - สร้าง RAG pipeline: ingestion → chunking → embeddings → vector storage → retrieval → grounded answer (มีการอ้างชิ้นส่วนข้อมูล/หลักฐาน) โดยใช้ embeddings ของ Gemini และ endpoint `embedContent` [6]
- สร้าง SQL RAG: ใช้ “function calling/tool use” ให้โมเดลสร้าง SQL ที่ปลอดภัย (read-only, จำกัดตาราง/คอลัมน์, LIMIT) แล้วรัน query จริงในฐานข้อมูลเพื่อนำผลมาสรุปตอบ [7]
- เพิ่ม Agentic RAG: routing (เลือกเครื่องมือ vector/SQL), critique/evaluate คุณภาพบริบท, query rewrite loop แบบมีเพดานรอบ (recursion/loop limit) ด้วย orchestration แบบกราฟ [8]
- Deploy: สาธิตแนวทางนำขึ้น Cloud Run (containerized) และเชื่อม Cloud SQL พร้อมการเก็บ secrets อย่างถูกต้อง และการป้องกันชั้นขอบเครือข่ายด้วย Cloud Armor [9]

เหตุผลเชิงสถาปัตยกรรม (ทำไมเลือกสิ่งนี้) - Google GenAI SDK เป็นไลบรารีที่เอกสาร Gemini แนะนำ, GA และเป็น "ทางหลัก" ในตัวอย่างเอกสารทางการ [10]
- `gemini-embedding-001` รองรับ output dimensionality แบบปรับได้ (MRL) และเอกสารระบุค่าที่แนะนำ 768/1536/3072 ซึ่งช่วยสอน trade-off ค่าใช้จ่าย/พื้นที่จัดเก็บ vs คุณภาพได้ตรงจุด [11]
- PostgreSQL + pgvector ช่วย "สอนทั้ง vector retrieval และ SQL retrieval ในฐานเดียว" เหมาะกับ workshop 2 วัน (ลดภาระ infra) และเอกสาร pgvector แจกแจง operator ระยะทาง (L2/cosine/inner product) ชัดเจน [12]

ตารางเวลาและภาพรวมหลักสูตร 2 วัน 5 เวิร์กช็อป

ภาพรวมเวลาอบรม: 2 วัน (แนะนำ 09:00–17:30/18:00)
Deliverable ปลายทาง: ผู้เรียนมี Web App Chatbot ที่ 1) ถาม–ตอบจากเอกสารด้วย RAG
2) ถาม–ตอบจากฐานข้อมูลด้วย SQL RAG
3) มี Agentic Routing/Validation และแนวทาง deploy

ตารางสรุปเวลา (ทั้งคลาส)

วัน	เวลา (โดยประมาณ)	กิจกรรมหลัก	ผลลัพธ์ที่ต้องได้
Day 1	09:00–09:15	เปิดคลาส/ตั้งเป้าหมาย/ตรวจระบบ	ทุกเครื่องพร้อมรันโค้ดและเรียก API
Day 1	09:15–12:15	Workshop 1: Foundations + Gemini RAG MVP	RAG แบบง่าย (in-memory) + เรียก Gemini ได้ [13]
Day 1	12:15–13:15	พักกลางวัน	—
Day 1	13:15–16:15	Workshop 2: Vector DB ด้วย Postgres + pgvector	สร้างตาราง/ดัชนี/ค้นหา cosine ได้ [12]
Day 1	16:15–17:30	Workshop 3 (Part A): Backend/Frontend Chat MVP	Chat UI + API + RAG retrieve ได้ (ยังไม่ agentic) [14]
Day 2	09:00–10:30	Workshop 3 (Part B): Streaming + Citations + Session	Chat streaming และยืนยันหลักฐาน (citations) [15]
Day 2	10:45–13:45	Workshop 4: SQL RAG ด้วย Function Calling	แชตบอทถามยอดขาย/สต็อก ฯลฯ จาก DB ได้ [16]
Day 2	14:45–17:45	Workshop 5: Agentic RAG + Deploy	Agent router/critique/rewrite + deploy Cloud Run [17]
Day 2	17:45–18:00	สอบ/สรุป/มอบการบ้าน	เกณฑ์ผ่าน + roadmap production

ตารางเปรียบเทียบเครื่องมือ/ไลบรารี (สรุปเพื่อเลือกใช้ในการสอน)

หมวด	ตัวเลือก	จุดเด่น	ข้อควรระวัง/ข้อเสีย	เหมาะเมื่อ
LLM/SDK	Gemini API + Google GenAI SDK	เอกสารทางการแนะนำ, GA, ตัวอย่างครบหลายภาษา, รองรับ function calling/tool-use และ embeddings ผ่าน `embedContent` [18]	ต้องจัดการ key/secrets และควบคุม prompt/tool อย่างรัดกุม	สอน end-to-end "RAG + tools" แบบอ้างอิงเอกสารทางการ
LLM/SDK (Production)	Vertex AI + Google GenAI SDK	แนวทางย้ายจาก Developer API ไป Vertex AI ได้โดยไม่ต้องเขียนใหม่มาก และขึ้นกับ IAM/ADC เหมาะกับองค์กร [19]	ต้องตั้งค่า project/location/สิทธิ์ IAM	สอน deployment / governance
Orchestration	LangGraph	นิยาม agent workflow เป็นกราฟ: state/nodes/edges และรองรับลูป/branch ชัดเจน เหมาะสอน agentic control [2]	ต้องออกแบบ state ให้ดี มิฉะนั้น debug ยาก	สอน Agentic RAG แบบมี routing/critique
Framework	LangChain	มีไกด์ RAG/SQL agent, มีแนวคิด "ตรวจ SQL/แก้ error loop" และเตือนเรื่องสิทธิ์ DB [20]	abstraction เยอะ อาจซ่อนรายละเอียด	สอนเร็ว/ใช้ของสำเร็จรูป
Framework	LlamaIndex	มีไกด์ Text-to-SQL + table/row/col retrieval และเตือนความเสี่ยงการรัน SQL ที่โมเดลสร้าง [21]	ผู้สอนต้องเลือก module ให้เหมาะ ไม่อย่างนั้นคอนเท็กซ์บวม	สอน advanced text-to-SQL retrieval
Vector DB	Postgres + pgvector	เก็บ vector "ร่วมกับข้อมูลอื่น" และมี operator ระยะทางชัดเจน (cosine `<=>`, L2 `<->`) [12]	ต้อง tune index/พารามิเตอร์เองเมื่อข้อมูลโตมาก	เวิร์กช็อป 2 วัน ทำให้ infra ง่าย
Vector DB	Weaviate	hybrid search = vector + keyword (BM25F) รวมผลได้ และปรับ weight (alpha) ได้ [22]	ต้องรัน service แยก/จัดการ cluster	สอน hybrid retrieval จริงจัง
Vector DB	Pinecone	มีแนวคิด namespaces และ metadata filters (ลดขอบเขตค้นหา) [23]	เป็นบริการภายนอก ต้องคุมต้นทุน/การเชื่อมต่อ	สอน SaaS vector DB production

แผนการสอนเชิงปฏิบัติการรายเวิร์กช็อป

หมายเหตุ: เพื่อให้ "พร้อมใช้สำหรับผู้ฝึกสอน" ทุกเวิร์กช็อปออกแบบให้มี **วัตถุประสงค์ชัด**, **ลำดับกิจกรรมแบบจับเวลา**, **สคริปต์การสอน**, **แบบฝึกหัดพร้อมเฉลย**, และ **แผ่นงาน**

Workshop 1: Foundations + Gemini RAG MVP (3:00 ชม.)

วัตถุประสงค์ - อธิบาย RAG/Embeddings/Prompt grounding และความต่างจาก "ถามโมเดลตรงๆ" ได้ - เรียก Gemini ผ่าน Google GenAI SDK เพื่อ generate และ embed ได้ตามเอกสารทางการ [13] - ทำ Mini-RAG แบบ in-memory เพื่อเห็นภาพก่อนย้ายลง vector DB

ลำดับกิจกรรม (จับเวลา) | ช่วงเวลา | นาที | กิจกรรม | สิ่งที่ผู้สอนต้องเน้น | |---:|---:|---|---| | 09:15–09:30 | 15 | Warm-up: "LLM ตอบมั่วได้อย่างไร" + ปัญหา private knowledge | ใช้ตัวอย่างเอกสารบริษัท/ข้อมูลภายใน | | 09:30–10:00 | 30 | Theory: RAG = retrieval + generation, embeddings คือเวกเตอร์แทนความหมาย | อธิบาย endpoint สำคัญ: `generateContent`, `streamGenerateContent`, `embedContent` [14] | | 10:00–10:20 | 20 | Setup: ติดตั้ง SDK, ตั้ง `GEMINI_API_KEY` | ชี้ว่าเอกสารแนะนำ `pip install google-genai` และตัวอย่างโค้ดทางการใช้ `genai.Client()` [24] | | 10:20–11:10 | 50 | Lab 1: Call Gemini + สร้าง embeddings | สอน `gemini-embedding-001` และ task types ที่เหมาะกับ retrieval (query/document) [25] | | 11:10–11:50 | 40 | Lab 2: In-memory retrieval + grounded prompt | ฝึก "ตอบโดยอิง context เท่านั้น" และใส่ citations id ชิ้นข้อมูล | | 11:50–12:15 | 25 | แบบฝึกหัด + เฉลย + wrap-up | เชื่อมไป workshop 2: ทำให้ retrieval "จริง" ด้วยฐานข้อมูล |

สคริปต์การสอน (ตัวอย่างที่ผู้สอนอ่านได้เลย) - เปิดประเด็น (2–3 นาที):
"วันนี้เราจะทำให้แชตบอท 'ไม่เดา' แต่ 'ค้นแล้วตอบ' โดยแยกบทบาทเป็น **Retriever** (ค้นหลักฐาน) และ **Generator** (เรียบเรียงคำตอบ)" - จุดย้ำที่ต้องพูดชัด (5 นาที):
"เอกสารทางการ Gemini ระบุ endpoint หลัก: สร้างข้อความแบบปกติ (`generateContent`), แบบสตรีม (`streamGenerateContent` ผ่าน SSE), และ embeddings (`embedContent`)—เราจะใช้ทั้งหมดใน 2 วัน" [14] - กับดักที่เตือน:
"ถ้าไม่บังคับให้โมเดลอ้างหลักฐาน มันจะเติมความรู้จากโลก/เดาเองได้—ดังนั้น prompt และ logic ต้องออกแบบให้ 'ตอบจาก context' เท่านั้น"

แบบฝึกหัด + เฉลย (Workshop 1) - แบบฝึกหัด 1: ปรับ chunk size แล้วบันทึกผล
- โจทย์: ลอง chunk = 300/600/900 คำ แล้วถามคำถามเดียวกัน 3 รอบ ดูว่า retrieval เปลี่ยนอย่างไร
- เฉลยแนวคิด: chunk เล็ก → context แคบ (อาจขาดบริบท), chunk ใหญ่ → context หนา (อาจดึงส่วนไม่เกี่ยว) ให้สรุป trade-off และเลือกค่าที่เหมาะกับ domain - แบบฝึกหัด 2: ออกแบบ prompt "ไม่รู้ต้องบอกไม่รู้"
- โจทย์: เพิ่มกติกา: ถ้า context ไม่มีคำตอบ ให้ตอบว่า "ไม่มีข้อมูลเพียงพอ" และถามคำถามกลับ 1 ข้อ
- เฉลยตัวอย่าง policy: "If you cannot find evidence in the provided context, say you don't know and ask for clarification."

แผ่นงานผู้เรียน (Worksheet 1: RAG Design Canvas) - Use case: ____
- แหล่งข้อมูลเอกสาร (PDF/FAQ/Policy): __
- ชนิดคำถามที่พบบ่อย*: ____*
- นิยาม "คำตอบที่ดี" ต้องมี: ( ) citations ( ) ตัวเลขจาก DB ( ) ขั้นตอน ( ) แหล่งอ้างอิง
- นโยบายเมื่อข้อมูลไม่พอ: _____
- ความเสี่ยงหลัก: ( ) hallucination ( ) prompt injection ( ) ข้อมูลลับหลุด

Workshop 2: Vector DB ด้วย PostgreSQL + pgvector (3:00 ชม.)

วัตถุประสงค์ - ตั้งค่า Postgres + pgvector และสร้าง schema สำหรับเก็บ embeddings - ทำ similarity search ด้วย cosine distance (`<=>`) และคำนวณ cosine similarity แบบ `1 - distance` ตามแนวทางในเอกสาร pgvector [12] - เข้าใจพื้นฐาน indexing (เช่น HNSW/IVFFlat) และเหตุผลว่าทำไมต้องมีเมื่อข้อมูลโตขึ้น (ระดับเวิร์กช็อปให้ "ใช้งานได้จริง" ไม่ลงลึกอัลกอริทึม)

ลำดับกิจกรรม (จับเวลา) | ช่วงเวลา | นาที | กิจกรรม | สิ่งที่ผู้สอนต้องเน้น | |---:|---:|---|---| | 13:15–13:35 | 20 | Theory: vector store vs keyword search | ผูกกับโจทย์ chatbot ที่ต้องค้น "ความหมาย" | | 13:35–14:10 | 35 |

Lab 1: สร้าง DB + เปิดใช้ `CREATE EXTENSION vector;` | อธิบายว่า pgvector เพิ่มชนิดข้อมูล `vector(n)` [12] | | 14:10–15:00 | 50 | Lab 2: ingest docs → embed → insert ลงตาราง doc_chunks | สอนแนวคิด metadata (doc_id, section, url, page) | | 15:00–15:30 | 30 | Lab 3: query top-k ด้วย cosine | อ้าง operator: `<=>` = cosine distance [12] | | 15:30–16:00 | 30 | Lab 4: สร้าง index (HNSW) + เปรียบเทียบเวลา | เน้น "ต้อง benchmark" และคุมพารามิเตอร์ | | 16:00–16:15 | 15 | Exercise + เฉลย | เตรียมไปสู่ web app |

สคริปต์การสอน - "วันนี้เราจะทำให้ RAG ของเรา 'ไม่จำกัดที่ in-memory' แต่กลายเป็นฐานความรู้ที่ query ได้จริง ด้วยการเก็บ embedding เป็นคอลัมน์ `vector(3072)` และค้นด้วย cosine distance operator `<=>`" [26]

แบบฝึกหัด + เฉลย (Workshop 2) - แบบฝึกหัด 1: เขียน SQL ให้คืน top_k พร้อมคะแนน similarity
- โจทย์: เขียน query คืน `chunk_id, content, cosine_similarity` โดยจัดเรียงจากมากไปน้อย
- เฉลยหลัก: ใช้ `1 - (embedding <=> :qvec) AS cosine_similarity` และ `ORDER BY embedding <=> :qvec ASC` แล้ว LIMIT k (เพราะ distance น้อย = คล้ายมาก) [12] - แบบฝึกหัด 2: เพิ่ม metadata filter
- โจทย์: จำกัดการค้นหาเฉพาะ doc_id = 'policy-hr'
- เฉลย: เพิ่ม `WHERE metadata->>'doc_id' = 'policy-hr'` ก่อน ORDER BY

แผ่นงานผู้เรียน (Worksheet 2: Ingestion Checklist) - ( ) เลือก embedding model และ dimension (768/1536/3072)
- ( ) กติกา chunking (ขนาด/overlap)
- ( ) metadata ที่ต้องเก็บ: doc_id/section/page/url
- ( ) คำสั่งทดสอบ retrieval (top_k=?, threshold=?)
- ( ) แผน re-embed เมื่อเปลี่ยนโมเดล (migration plan)

Workshop 3: Web App Chatbot (Backend + Frontend) (รวม 2:30 ชม.)

วัตถุประสงค์ - ทำ API สำหรับ chat (backend) + UI (frontend) ที่เรียก RAG retrieval และแสดง citations - เพิ่ม streaming เพื่อ UX ที่ดีขึ้น โดยอิงแนวคิด streaming/SSE ที่เอกสาร Gemini API ระบุ (`streamGenerateContent`) [14] - จัดการ conversation state แบบเบาๆ (session id + short memory)

ลำดับกิจกรรม (จับเวลา) | ช่วงเวลา | นาที | กิจกรรม | จุดเน้น | |---:|---:|---|---| | Day1 16:30–17:30 | 60 | Part A: ทำ backend `/chat` แบบ non-stream | แยกชั้น: controller → retrieval → prompt → LLM | | Day2 09:00–09:30 | 30 | Part B1: ปรับ UI ให้แสดง citations | UI ต้องโชว์ "หลักฐานที่ใช้" | | Day2 09:30–10:30 | 60 | Part B2: streaming response | อธิบายว่ามีทั้ง streaming ใน Gemini API และการรองรับ streaming ใน Cloud Run (แนวคิด SSE) [15] |

สคริปต์การสอน - "ภาคนี้เราจะเปลี่ยนจาก 'โน้ตบุ๊ก' เป็น 'โปรดักต์' โดยทำให้ผู้ใช้พิมพ์คำถามแล้วเห็นคำตอบทันที พร้อมลิงก์หลักฐาน (chunk id / แหล่งเอกสาร)"

แบบฝึกหัด + เฉลย (Workshop 3) - แบบฝึกหัด 1: เพิ่ม session memory แบบสรุป
- โจทย์: ถ้าประวัติแชตยาว ให้สร้าง "summary" เก็บแทน และส่งให้โมเดลเป็นบริบทแทน log ทั้งหมด
- เฉลยแนวคิด: ทำ endpoint `/summarize` หรือทำ summary ทุก N turns แล้วเก็บใน server-side session store - แบบฝึกหัด 2: ทำ "ตอบไม่เกิน 6 บรรทัด + bullet"
- โจทย์: ปรับ system prompt ให้ตอบกระชับและขึ้น bullet เฉพาะ key points
- เฉลย: เพิ่ม formatting constraints ใน system instruction

แผ่นงานผู้เรียน (Worksheet 3: API Contract) - `/chat` request schema: message, session_id, mode (rag/sql/auto), top_k

- `/chat` response schema: answer_text, citations[], latency_ms, tool_trace[]
- acceptance criteria: "ต้องมี citations อย่างน้อย 1 รายการเมื่อ mode=rag"

Workshop 4: SQL RAG (Text-to-SQL + Tool Use) (3:00 ชม.)

วัตถุประสงค์ - ใช้ function calling ให้โมเดลตัดสินใจเรียก "เครื่องมือ query_sql" เมื่อผู้ใช้ถามข้อมูลเชิงตัวเลข/ตาราง - สร้าง guardrails: อ่านอย่างเดียว, จำกัดตาราง, บังคับ LIMIT, ป้องกัน DML - สังเคราะห์คำตอบจากผล query จริง (ไม่เดา)

ฐานแนวคิดจากเอกสารทางการ - Gemini API อธิบายว่า function calling ใช้เพื่อ "Augment Knowledge" จากแหล่งภายนอก เช่น DB/API [27]
- เอกสาร Vertex AI function calling ย้ำแนวปฏิบัติ: ใช้ structured output/validate API calls/ใช้ temperature ต่ำเพื่อความ deterministic (ในบริบท function calling) [28]
- เอกสารตัวอย่างจาก LangChain และ LlamaIndex เตือนตรงกันว่า "การรัน SQL ที่โมเดลสร้างมีความเสี่ยง" ต้องใช้สิทธิ์จำกัด/DB read-only/sandbox [29]

ลำดับกิจกรรม (จับเวลา) | ช่วงเวลา | นาที | กิจกรรม | จุดเน้น | |---:|---:|---|---| | 10:45–11:15 | 30 | Theory: เมื่อไรควรใช้ SQL vs RAG | SQL = ตัวเลข/สรุป/agg; RAG = policy/procedure | | 11:15–12:15 | 60 | Lab 1: สร้าง schema ตัวอย่าง (sales DB) + seed data | ตาราง customers/orders/order_items/products | | 12:15–13:00 | 45 | Lab 2: สร้าง `query_sql` tool + validation rules | forbid DML, allowlist tables, inject LIMIT | | 13:00–13:30 | 30 | Lab 3: ให้โมเดลสร้าง SQL ผ่าน function calling และตอบจากผลจริง | แยก "SQL generation" กับ "answer synthesis" | | 13:30–13:45 | 15 | Exercise + เฉลย | เพิ่มกรณีคลุมเครือให้ถามกลับ |

สคริปต์การสอน - "ข้อความสำคัญวันนี้: ถ้าผู้ใช้ถาม 'ยอดขายเดือนนี้เท่าไร' แล้วระบบไม่ query DB จริง นั่นไม่ใช่ SQL RAG—มันคือการเดา" - "ข้อกำหนดด้านความปลอดภัย: ใช้ DB user แบบ read-only, จำกัด schema, และ validate ทุก query ก่อนรัน" [30]

แบบฝึกหัด + เฉลย (Workshop 4) - แบบฝึกหัด 1: ป้องกัน query ที่ไม่มี LIMIT
- โจทย์: ถ้าโมเดลไม่ใส่ LIMIT ให้ระบบเติม LIMIT 100 อัตโนมัติ
- เฉลย: validation middleware ตรวจ regex `\blimit\b` หากไม่พบให้ append `LIMIT 100` - แบบฝึกหัด 2: คำถามคลุมเครือ
- โจทย์: "ยอดขายสูงสุด" ต้องถาม "ช่วงเวลาไหน/นับจากยอดเงินหรือจำนวนออเดอร์"
- เฉลย: ให้ router node บังคับถามกลับ (clarifying question) ก่อนอนุญาตให้รัน SQL

แผ่นงานผู้เรียน (Worksheet 4: SQL Safety Checklist) - DB account: ( ) read-only ( ) จำกัด schema ( ) row-level security (optional)
- Query allowlist: tables = ___
- Forbidden keywords: INSERT/UPDATE/DELETE/DROP/ALTER/CREATE
- Max rows: _ / Max runtime: _____
- Logging: เก็บ ( ) user question ( ) SQL ( ) row count ( ) latency

Workshop 5: Agentic RAG (Router + Critique + Tool Use) + Deploy (3:00 ชม.)

วัตถุประสงค์ - สร้าง "Agentic RAG" ที่เลือกแหล่งข้อมูล (vector/SQL) อัตโนมัติ, ตรวจคุณภาพบริบท, และวนแก้ query เมื่อจำเป็น - ใช้ orchestration แบบกราฟ (state/nodes/edges) เพื่อควบคุมลูป/branch อย่างชัดเจน [2]
- ทำ deployment ที่พร้อมใช้งานจริง: secrets, least privilege, scaling, edge protection

แนวคิดเวิร์กโฟลว์ Agentic RAG - อ้างอิงเอกสารประกอบที่ผู้ใช้ให้มา: ไฟล์ "Agentic RAG Workflows Explained.pdf"
- ประกอบกับหลักการ tool use/function calling ที่เชื่อมโมเดลเข้ากับเครื่องมือภายนอกได้ [28]

ลำดับกิจกรรม (จับเวลา) | ช่วงเวลา | นาที | กิจกรรม | จุดเน้น | |---:|---:|---|---| | 14:45–15:20 | 35 | Theory: Agentic patterns (routing/critique/parallel/sequential) | “ควบคุม agency” และตั้ง loop limit | | 15:20–16:20 | 60 | Lab 1: สร้าง graph: route → retrieve_docs/query_sql → evaluate → answer | state schema + node functions | | 16:20–17:00 | 40 | Lab 2: เพิ่ม self-check + query rewrite | ใช้ judge rubric แบบสั้น (0–2) | | 17:00–17:45 | 45 | Lab 3: Deploy บน Cloud Run + เชื่อม Cloud SQL + Secrets | แนะนำ Secret Manager + policy [31] |

สคริปต์การสอน - “Agentic RAG ไม่ใช่แค่ ‘มี tools’ แต่เป็น ‘มีการตัดสินใจหลายขั้น’ และ ‘มีการตรวจสอบ’ ก่อนตอบ” - “ความปลอดภัยของ agent: ต้องคุม ‘excessive agency’ และ prompt injection ตามแนวคิดความเสี่ยง LLM ที่ OWASP สรุปไว้” [32]
- “ถ้าจะ production ให้พิจารณาใช้ตัวกรอง/การป้องกัน prompt injection ระดับ runtime เช่น Model Armor” [33]

แบบฝึกหัด + เฉลย (Workshop 5) - แบบฝึกหัด 1: จำกัดรอบลูป (loop limit)
- โจทย์: หาก evaluate แล้ว “ไม่ผ่าน” ให้ rewrite ได้ไม่เกิน 2 ครั้ง แล้วจบด้วย “ขอข้อมูลเพิ่ม”
- เฉลย: ตั้ง `max_iterations=2` ใน state และทุกครั้งที่ rewrite เพิ่ม counter; เกินแล้วตอบคำถามกลับ - แบบฝึกหัด 2: เพิ่มนโยบาย “ห้ามตอบนอกหลักฐาน” แบบบังคับ
- โจทย์: ถ้าทั้ง vector และ SQL ไม่ให้หลักฐาน ให้ตอบ “ไม่มีข้อมูล”
- เฉลย: final node เช็ก `evidence_count==0` แล้ว override response

แผ่นงานผู้เรียน (Worksheet 5: Agent Flow Design) - Intent categories: ( ) SQL ( ) Docs ( ) Mixed ( ) Unknown
- Tools available: ( ) vector_search ( ) query_sql ( ) none
- Evaluate rubric: relevance(0–2), sufficiency(0–2), risk(0–2)
- Loop limit: __
- Fallback response: _____

ตัวอย่างสถาปัตยกรรมและโค้ดอ้างอิงสำหรับ Web App Chatbot

ไดอะแกรมสถาปัตยกรรม RAG Web App (Mermaid)

```
flowchart LR
  U[User (Browser)] -->|Chat message| FE[Frontend Web App]
  FE -->|HTTPS /chat| BE[Backend API (FastAPI)]
  BE -->|embedContent| EMB[Gemini Embeddings]
  BE -->|vector query| VDB[(PostgreSQL + pgvector)]
  BE -->|SQL query (read-only)| SQLDB[(PostgreSQL tables)]
  BE -->|generateContent / streamGenerateContent| LLM[Gemini LLM]
  VDB -->|top-k chunks + metadata| BE
  SQLDB -->|rows/aggregates| BE
  BE -->|answer + citations + tool trace| FE
```

หมายเหตุที่ผูกกับเอกสารทางการ - Gemini API ระบุ endpoint หลักสำหรับ generation/streaming/embeddings ชัดเจน (รวมถึง streaming ผ่าน SSE) [14]
- Embeddings ของ `gemini-embedding-001` มีตัวเลือก `output_dimensionality` และมีคำแนะนำ dimension ที่ใช้บ่อย (768/1536/3072) [11]
- pgvector ระบุการเปิดใช้ extension (`CREATE EXTENSION vector;`) และ operator distance เช่น cosine `<=>` [12]

Flowchart ของ Agentic RAG (Router + Critique + Rewrite) (Mermaid)

```
flowchart TD
  A[User Question] --> B[Route Intent]
  B -->|SQL| C[Generate SQL via Function Calling]
  B -->|Docs| D[Vector Retrieval]
  B -->|Mixed| E[Parallel: SQL + Vector]
  C --> F[Validate SQL 정책]
  F -->|Pass| G[Execute SQL (read-only)]
  F -->|Fail| H[Ask Clarifying Question / Refuse]
  D --> I[Evaluate Context Quality]
  G --> J[Evaluate Evidence Sufficiency]
  E --> I
  E --> J
  I -->|Low| K[Rewrite Query / Expand Retrieval]
  J -->|Low| K
  K -->|Loop < limit| B
  K -->|Loop >= limit| H
  I -->|Good| L[Generate Answer with Citations]
  J -->|Good| L
  L --> M[Return Answer + Citations + Trace]
```

หลักการสำคัญที่อ้างอิงได้ - Function calling เป็น “สะพาน” เชื่อมโมเดลกับเครื่องมือ/ระบบจริง และเอกสารแนะนำให้ validate เมื่อมีผลกระทบสำคัญ [34]
- LangGraph อธิบายการสร้าง workflow ด้วย state/nodes/edges เพื่อทำ branching/looping เป็นระบบ [2]
- ความเสี่ยงหลักของ agentic systems รวม prompt injection/excessive agency ถูกสรุปเป็นหมวดความเสี่ยงใน OWASP Top 10 for LLM Applications [32]

โครงสร้างโปรเจกต์ตัวอย่าง (แนะนำให้ผู้สอนใช้เป็น “starter kit”)

```
rag-chatbot-workshop/
  backend/
    main.py
    config.py
    rag/
      ingest.py
      retriever_pgvector.py
      prompts.py
    sql/
      schema.sql
      seed.sql
      sql_tool.py
    agent/
      graph.py
  frontend/
    src/App.tsx
    src/api.ts
  infra/
```

```
    docker-compose.yml
    init_pgvector.sql
```

ตัวอย่างโค้ด (ย่อ) ที่ครอบคลุม backend, RAG pipeline, SQL retrieval, vector DB, prompt templates

1) Backend config (Python)

```python
# backend/config.py
import os
from dataclasses import dataclass

@dataclass(frozen=True)
class Settings:
  gemini_api_key: str = os.getenv("GEMINI_API_KEY", "")
  db_url: str = os.getenv("DATABASE_URL", "postgresql+psycopg://
app:app@localhost:5432/appdb")
  embedding_model: str = os.getenv("EMBED_MODEL", "gemini-embedding-001")
  chat_model: str = os.getenv("CHAT_MODEL", "gemini-2.5-flash") # เปลี่ยนได้ตามสิทธิ์ใน
โปรเจกต์
  embed_dim: int = int(os.getenv("EMBED_DIM", "3072"))
  top_k: int = int(os.getenv("TOP_K", "5"))

settings = Settings()
```

เอกสาร Gemini API ระบุการติดตั้ง SDK และการใช้ `genai.Client()` ที่อ่าน key จาก `GEMINI_API_KEY` [24]

2) Prompt templates (RAG + SQL)

```python
# backend/rag/prompts.py
RAG_SYSTEM = """คุณเป็นผู้ช่วยตอบคำถามจากฐานความรู้ภายในองค์กร
กติกา:
- ตอบจาก CONTEXT เท่านั้น
- ทุกคำตอบต้องอ้างอิงหลักฐานเป็น [CIT#] อย่างน้อย 1 จุด
- ถ้า CONTEXT ไม่พอ ให้ตอบว่า "ไม่มีข้อมูลเพียงพอ" และถามคำถามกลับ 1 ข้อ
"""

RAG_USER = """คำถาม: {question}

CONTEXT:
{context}

ให้ตอบเป็นภาษาไทย กระชับ และใส่ [CIT#] ต่อท้ายประโยคที่ใช้หลักฐาน
"""

SQL_SYSTEM = """คุณเป็นผู้ช่วยที่สร้าง SQL แบบอ่านอย่างเดียว (READ ONLY)
กติกา:
```

```
- ห้ามใช้ INSERT/UPDATE/DELETE/DROP/ALTER/CREATE
- ต้องมี LIMIT เสมอ (ถ้าไม่ระบุให้ใช้ LIMIT 100)
- ใช้เฉพาะตารางที่อนุญาต: {allowed_tables}
- ถ้าคำถามคลุมเครือ ให้ถามกลับก่อน
"""
```

ตัวอย่าง prompt แนว SQL agent ในเอกสาร LangChain ย้ำ “ห้าม DML” และ “ต้อง LIMIT” [35]

3) สร้าง embeddings + ingest ลง pgvector

```
# backend/rag/ingest.py
from google import genai
from google.genai.types import EmbedContentConfig
from sqlalchemy import text
from sqlalchemy.engine import Engine

def embed_texts(texts: list[str], model: str, output_dim: int) ->
list[list[float]]:
  client = genai.Client()
  resp = client.models.embed_content(
    model=model,
    contents=texts,
    config=EmbedContentConfig(
      task_type="RETRIEVAL_DOCUMENT",
      output_dimensionality=output_dim,
    ),
  )
  return [e.values for e in resp.embeddings]

def ingest_chunks(engine: Engine, chunks: list[dict], embed_model: str,
output_dim: int):
  texts = [c["content"] for c in chunks]
  vectors = embed_texts(texts, embed_model, output_dim)

  with engine.begin() as con:
    for c, v in zip(chunks, vectors):
      con.execute(
        text("""
                    INSERT INTO doc_chunks(chunk_id, doc_id, chunk_index,
content, metadata, embedding)
                    VALUES
(:chunk_id, :doc_id, :chunk_index, :content, :metadata::jsonb, :embedding)
                  """),
        {
          "chunk_id": c["chunk_id"],
          "doc_id": c["doc_id"],
          "chunk_index": c["chunk_index"],
          "content": c["content"],
          "metadata": c["metadata_json"],
```

```
        # หมายเหตุ: driver บางตัวต้องแปลงรูปแบบ embedding ให้ถูกก่อน insert
        "embedding": v,
      },
    )
```

เอกสาร Embeddings ของ Gemini อธิบาย `gemini-embedding-001`, `output_dimensionality` และ task_type สำหรับ retrieval [25]

4) pgvector schema + index (SQL)

```
-- backend/sql/schema.sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS doc_chunks (
 chunk_id   uuid PRIMARY KEY,
 doc_id    text NOT NULL,
 chunk_index  int  NOT NULL,
 content    text NOT NULL,
 metadata   jsonb NOT NULL DEFAULT '{}'::jsonb,
 embedding   vector(3072) NOT NULL
);

-- ตัวอย่าง index (เลือกแบบที่ทีมสอนถนัด)
-- หมายเหตุ: ผู้สอนควรสอนการ benchmark และค่อยตัดสินใจเลือก
-- CREATE INDEX ON doc_chunks USING hnsw (embedding vector_cosine_ops);
```

pgvector ระบุการเปิดใช้ extension (`CREATE EXTENSION vector;`) และสนับสนุน cosine distance (`<=>`) [12]

5) Retriever ด้วย cosine distance `<=>`

```
# backend/rag/retriever_pgvector.py
from sqlalchemy import text
from sqlalchemy.engine import Engine

def retrieve_top_k(engine: Engine, qvec: list[float], k: int) -> list[dict]:
  # pgvector: cosine distance = <=>  และ cosine_similarity = 1 - distance
  rows = []
  with engine.begin() as con:
    res = con.execute(
      text("""
              SELECT
                chunk_id,
                content,
                metadata,
                1 - (embedding <=> :qvec) AS cosine_similarity
              FROM doc_chunks
              ORDER BY embedding <=> :qvec
```

```
                LIMIT :k
            """),
      {"qvec": qvec, "k": k},
    )
    for r in res:
      rows.append(dict(r._mapping))
  return rows
```

เอกสาร pgvector อธิบาย `<=>` เป็น cosine distance และสูตร cosine similarity = `1 - cosine distance` [12]

6) SQL tool + validation (แกนของ SQL RAG)

```
# backend/sql/sql_tool.py
import re
from sqlalchemy import text
from sqlalchemy.engine import Engine

FORBIDDEN = re.compile(r"\b(insert|update|delete|drop|alter|create)\b", re.I)

def validate_sql(sql: str) -> str:
  if FORBIDDEN.search(sql):
    raise ValueError("Forbidden SQL keyword detected (DML/DDL not allowed).")
  if "limit" not in sql.lower():
    sql = sql.rstrip().rstrip(";") + " LIMIT 100"
  return sql

def run_readonly_sql(engine: Engine, sql: str) -> list[dict]:
  sql = validate_sql(sql)
  with engine.begin() as con:
    res = con.execute(text(sql))
    cols = res.keys()
    return [dict(zip(cols, row)) for row in res.fetchall()]
```

ทั้ง LangChain และ LlamaIndex เตือนความเสี่ยงของ text-to-SQL และแนะนำใช้ role จำกัด/DB read-only/การป้องกันเพิ่มเติม [29]

7) Function calling (ย่อ) เพื่อให้โมเดลเรียก `query_sql` (manual loop)

```
# backend/sql/function_calling_sql.py
from google import genai
from google.genai import types

def make_query_sql_decl():
  return {
    "name": "query_sql",
    "description": "Run a READ-ONLY SQL query and return rows as JSON.",
    "parameters": {
```

```
      "type": "OBJECT",
      "properties": {
        "sql": {"type": "STRING", "description": "READ-ONLY SQL with LIMIT."}
      },
      "required": ["sql"]
    }
  }

def ask_model_for_sql(question: str, system: str):
  client = genai.Client()
  tools = [types.Tool(function_declarations=[make_query_sql_decl()])]
  config = types.GenerateContentConfig(tools=tools)
  contents = [types.Content(role="user", parts=[types.Part(text=f"{system}
\n\n{question}")])]
  resp = client.models.generate_content(model="gemini-2.5-flash",
contents=contents, config=config)
  return resp
```

เอกสาร Gemini API function calling แสดงรูปแบบการประกาศ `Tool(function_declarations=...)` และการส่ง `GenerateContentConfig(tools=...)` [36]

8) Frontend (React) โค้ดย่อ: ส่งคำถามและแสดง citations

```
// frontend/src/App.tsx
import { useState } from "react";

type ChatResp = { answer_text: string; citations: { id: string; title?: string }[] };

export default function App() {
 const [msg, setMsg] = useState("");
 const [log, setLog] = useState<string[]>([]);
 const [cits, setCits] = useState<ChatResp["citations"]>([]);

 async function send() {
  const r = await fetch("/chat", {
   method: "POST",
   headers: { "Content-Type": "application/json" },
   body: JSON.stringify({ message: msg, session_id: "demo" }),
  });
  const data: ChatResp = await r.json();
  setLog((x) => [...x, `Q: ${msg}`, `A: ${data.answer_text}`]);
  setCits(data.citations);
  setMsg("");
 }

 return (
  <div style={{ padding: 16 }}>
   <h2>RAG Chatbot</h2>
```

```
    <textarea value={msg} onChange={(e) => setMsg(e.target.value)} rows={3} />
    <button onClick={send}>Send</button>

    <pre>{log.join("\n")}</pre>

    <h3>Sources</h3>
    <ul>
     {cits.map((c) => (
      <li key={c.id}>{c.title ?? c.id}</li>
     ))}
    </ul>
   </div>
  );
}
```

การนำไปใช้งานจริง การประเมินผล และแหล่งอ้างอิงทางการ

ข้อกำหนดซอฟต์แวร์/ฮาร์ดแวร์ (สำหรับผู้สอนเตรียมก่อนวันอบรม)

ขั้นต่ำต่อเครื่องผู้เรียน - CPU 4 cores+, RAM 16GB+, SSD ว่าง 10GB
- OS: Windows/macOS/Linux
- Python 3.9+ (ตาม quickstart) และ Node.js 18+ (ตาม quickstart) [37]
- Docker Desktop (เพื่อรัน Postgres+pgvector ง่าย)
- Git และ editor (VS Code)

บัญชี/สิทธิ์ - Gemini API key ผ่าน environment variable `GEMINI_API_KEY` ตามเอกสาร [38]
- (ทางเลือก production) GCP Project + Vertex AI API enabled + ADC และ env vars ของ GenAI SDK ตามเอกสาร Vertex AI [19]

วิธีประเมินผลและเกณฑ์ผ่าน

องค์ประกอบคะแนน (แนะนำ) - Lab checkpoints (Workshop 1–5): 50%
- Mini project (ปลาย Day 2): 40%
- Quiz สั้นเรื่อง security + prompt/tool rules: 10% (อิง OWASP/แนวทาง validate tool calls) [39]

เกณฑ์ผ่าน (Pass criteria) - คะแนนรวม ≥ 70%
- และต้องผ่าน “Functional Tests” 5 ข้อ (ผ่าน/ไม่ผ่าน) 1) `/chat` ตอบจากเอกสารและมี citations อย่างน้อย 1 รายการ เมื่อถามคำถามที่มีคำตอบใน corpus
2) ถ้าคำตอบไม่มีใน corpus ต้องตอบว่า “ไม่มีข้อมูลเพียงพอ” + ถามกลับ 1 ข้อ
3) SQL RAG ตอบคำถามเชิงตัวเลขได้จาก DB จริง และ query เป็น read-only
4) Agentic routing เลือก SQL เมื่อถามยอดขาย และเลือก RAG เมื่อถาม policy
5) ระบบมี loop limit และไม่วนไม่จบ (ควบคุม recursion)

ตัวอย่าง deployment สำหรับ Web App (hosting, security, scaling)

สถาปัตยกรรมแนะนำ (เริ่มต้น) - Backend: Cloud Run (container)
- Database: Cloud SQL for PostgreSQL
- Secrets: Secret Manager (mount เป็น env หรือ file)

- Edge protection: Cloud Armor (WAF/DDoS) ผ่านการตั้งค่าให้ Cloud Run อยู่หลัง load balancer (แนวทางในเอกสาร Cloud Run/Cloud Armor) [40]
- Frontend: จะ host ที่ไหนก็ได้ (เช่น Cloud Run static, หรือบริการ hosting อื่น) — ในเวิร์กช็อปให้เน้นที่หลักการ

Security ที่ต้องสอน "เป็นขั้นตอน" 1) Secrets - Cloud Run ตรวจสิทธิ์ secret ระหว่าง deploy/runtime และต้องให้ service account ของ Cloud Run มีสิทธิ์เข้าถึง secret [41]
- แนวปฏิบัติ Secret Manager: แยก secret ตาม environment, จำกัด IAM, rotation ตามความเหมาะสม [42]

2) DB connectivity - เอกสาร Cloud SQL ระบุแนวทางการ connect จาก Cloud Run พร้อมตัวอย่าง [43]

3) Network protection - เอกสาร Cloud Run ระบุสามารถใช้ Cloud Armor เพื่อป้องกัน Cloud Run จากภัย DDoS/แอปโจมตี (เช่น XSS/SQLi) [44]

4) LLM/Agent security - ใช้แนวปฏิบัติ OWASP Top 10 for LLM Applications เพื่อสื่อสารความเสี่ยง (Prompt Injection ฯลฯ) [32]
- หากต้องการชั้นป้องกันเพิ่มเติมในองค์กร: พิจารณา Model Armor สำหรับ screening prompt/response และลดความเสี่ยง prompt injection/sensitive data leaks [45]

Scaling ที่ผู้สอนควรอธิบาย - Cloud Run ปรับ concurrency ได้ถึง 1000 และมีค่า default (เชื่อมกับจำนวน vCPU) [46]
- Autoscaling พิจารณา CPU utilization, concurrency และ max/min instances [47]
- สอน "ผลกระทบต่อ DB": ถ้า scale out มากอาจเปิด connection เยอะ → ต้องมี connection pooling และตั้ง max instances ให้เหมาะสม [48]

แผนการบ้าน (เพื่อปิดช่องว่างหลังอบรม)

การบ้านหลัง Day 1 (ใช้เวลา 1–2 ชม.) - เพิ่มเอกสารใหม่ 1 ชุด (เช่น FAQ ภายใน) และรัน ingest → ทดสอบ retrieval top_k 3/5/10
- ออกแบบ prompt ที่บังคับ citations และตอบไม่เกิน 8 บรรทัด

การบ้านหลัง Day 2 (ใช้เวลา 2–4 ชม.) - ปรับ router ให้รองรับ "Mixed question" (ต้องใช้ SQL + RAG ในคำตอบเดียว)
- ทำ test set 20 คำถาม (10 docs, 10 SQL) + วัด failure cases
- ทดลอง deploy ขึ้น Cloud Run และเก็บ secrets ด้วย Secret Manager [49]

แหล่งอ้างอิงลิงก์หลัก (เอกสารทางการ/เชิงอ้างอิง) พร้อมวันที่อ้างอิง

> หมายเหตุการอ้างอิง: รายการนี้ตั้งใจให้เป็น "ลิงก์หลัก" สำหรับผู้ฝึกสอนเปิดระหว่างสอน และทุกหน้าที่เป็น Gemini API docs มีบรรทัด "Last updated" ซึ่งในรายงานนี้อ้างอิงตามที่หน้าเอกสารแสดง และถือว่าเข้าถึง ณ วันที่ 23 ก.พ. 2026 (Asia/Bangkok)

Gemini (ทางการ) - Gemini API Quickstart — Last updated 2026-01-19 UTC [37]
- Gemini API Libraries (Google GenAI SDK) — มีสถานะ GA และมีคำเตือนเรื่อง legacy SDK; Last updated 2026-01-19 UTC [50]
- Gemini API Reference (generate/stream/embed) — Last updated 2026-01-19 UTC [14]
- Gemini API Embeddings (`gemini-embedding-001`, output_dimensionality, task types) — Last updated 2026-01-19 UTC [25]
- Gemini API Function Calling (best practices/tool config/allowed functions) — Last updated 2026-02-20 UTC [51]

Google Cloud / Vertex AI (ทางการ) - Google Gen AI SDK on Vertex AI (unified interface + migrate path) [52]
- Vertex AI Text Embeddings API + รายชื่อโมเดล (รวม `gemini-embedding-001`) [53]
- Vertex AI "Get text embeddings" (ระบุ 3072 dims และ normalized vectors) — Last updated 2026-02-20 UTC [54]
- Vertex AI Function Calling (tool use; validate calls; structured output) [55]
- Vertex AI RAG Engine overview/quickstart (ตัวเลือก managed RAG) — Last updated 2026-02-20 UTC [56]

Vector DBs (เอกสารหลัก) - pgvector README (operators `<=>`, cosine similarity, การเปิดใช้ extension) [12]
- Weaviate hybrid search (vector + BM25F) และการปรับ alpha [22]
- Pinecone filter by metadata (แนวคิด metadata filtering) [57]

SQL RAG libraries (แนวทาง/คำเตือนสำคัญ) - LangChain: Build a SQL agent (ขั้นตอน agent และคำเตือนเรื่องสิทธิ์ DB) [35]
- LlamaIndex: Text-to-SQL Guide (มีคำเตือน security และแนวคิด table/row/col retrieval) [21]

Deployment & Security (Google Cloud) - Cloud Run: Configure secrets (ตรวจสิทธิ์ secrets ตอน deploy/runtime) [41]
- Secret Manager best practices [42]
- Cloud SQL: Connect from Cloud Run [43]
- Cloud Run concurrency [46]
- Cloud Run autoscaling [47]
- Cloud Run + Cloud Armor [40]

แหล่งภาพ/ไดอะแกรมออนไลน์ที่แนะนำ (เปิดจาก citation ได้) - ภาพ "Vertex AI RAG key concepts" ในหน้า RAG Engine overview [58]
- ภาพ "Function Calling Interaction flow" ในหน้า Vertex AI function calling [55]
- บทความ Cloud Run streaming (มีไดอะแกรมแนวคิด streaming/SSE) [59]

---

[1] [10] [18] [50] https://ai.google.dev/gemini-api/docs/libraries
https://ai.google.dev/gemini-api/docs/libraries

[2] [8] [17] https://docs.langchain.com/oss/python/langgraph/graph-api
https://docs.langchain.com/oss/python/langgraph/graph-api

[3] [31] [41] [49] Configure secrets for services | Cloud Run
https://docs.cloud.google.com/run/docs/configuring/services/secrets?utm_source=chatgpt.com

[4] [5] [13] [24] [37] [38] https://ai.google.dev/gemini-api/docs/quickstart
https://ai.google.dev/gemini-api/docs/quickstart

[6] [11] [25] https://ai.google.dev/gemini-api/docs/embeddings
https://ai.google.dev/gemini-api/docs/embeddings

[7] [16] [27] [34] [36] [51] https://ai.google.dev/gemini-api/docs/function-calling
https://ai.google.dev/gemini-api/docs/function-calling

[9] [43] Connect from Cloud Run | Cloud SQL for PostgreSQL
https://docs.cloud.google.com/sql/docs/postgres/connect-run?utm_source=chatgpt.com

[12] [26] https://github.com/pgvector/pgvector
https://github.com/pgvector/pgvector

14 15 https://ai.google.dev/api
https://ai.google.dev/api

19 52 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/sdks/overview
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/sdks/overview

20 29 30 35 https://docs.langchain.com/oss/python/langchain/sql-agent
https://docs.langchain.com/oss/python/langchain/sql-agent

21 https://developers.llamaindex.ai/python/examples/index_structs/struct_indices/sqlindexdemo/
https://developers.llamaindex.ai/python/examples/index_structs/struct_indices/sqlindexdemo/

22 https://docs.weaviate.io/weaviate/search/hybrid
https://docs.weaviate.io/weaviate/search/hybrid

23 57 https://docs.pinecone.io/guides/search/filter-by-metadata
https://docs.pinecone.io/guides/search/filter-by-metadata

28 55 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling

32 39 https://owasp.org/www-project-top-10-for-large-language-model-applications/
https://owasp.org/www-project-top-10-for-large-language-model-applications/

33 https://cloud.google.com/security/products/model-armor
https://cloud.google.com/security/products/model-armor

40 44 Use Google Cloud Armor | Cloud Run
https://docs.cloud.google.com/run/docs/securing/cloud-armor?utm_source=chatgpt.com

42 Secret Manager best practices
https://docs.cloud.google.com/secret-manager/docs/best-practices?utm_source=chatgpt.com

45 https://docs.cloud.google.com/model-armor/overview
https://docs.cloud.google.com/model-armor/overview

46 Maximum concurrent requests for services | Cloud Run
https://docs.cloud.google.com/run/docs/about-concurrency?utm_source=chatgpt.com

47 About instance autoscaling in Cloud Run services
https://docs.cloud.google.com/run/docs/about-instance-autoscaling?utm_source=chatgpt.com

48 Set maximum instances for services | Cloud Run
https://docs.cloud.google.com/run/docs/configuring/max-instances?utm_source=chatgpt.com

53 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/text-embeddings-api
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/text-embeddings-api

54 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings

56 58 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/rag-overview
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/rag-overview

59 Cloud Run now supports HTTP/gRPC server streaming
https://cloud.google.com/blog/products/serverless/cloud-run-now-supports-http-grpc-server-streaming?utm_source=chatgpt.com